



PYTHON INTERVIEW NOTES



TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



① What is Python?

Answer: Python is a high-level, interpreted, object-oriented programming language. It is easy to learn, has simple syntax and a vast standard library.

② What are the key features of Python?

Answer:

- Easy to read and write
- Interpreted language
- Object-oriented
- Platform independent
- Open source
- Large standard library
- Supports multiple programming paradigms (OOP, Functional, Procedural)

③ What is the difference between a compiled language and an interpreted language?

Compiled Language	Interpreted Language
• Converts entire program into machine code before execution. • Executes faster. • Examples: C, C++	• Executes code line by line. • Easier to debug. • Examples: Python

④ What are variables in Python?

Answer: A variable is a name used to store data in memory. Python automatically determines the data type based on the assigned value.

```
name = "Mounika"
age = 22
cgpa = 8.5
is_student = True
```

⑤ What are the built-in data types in Python?

Answer: `int` `float` `str` `bool` `list` `tuple` `set` `dict` `complex`

⑥ What is the difference between a List and a Tuple?

List	Tuple
• Mutable (can be changed) • Uses square brackets []	• Immutable (cannot be changed) • Uses parentheses ()
Example: <code>my_list = [1, 2, 3]</code>	Example: <code>my_tuple = (1, 2, 3)</code>



⑦ What is the difference between '=' and '=='?

Answer:

- '=' is the assignment operator → assigns a value to a variable.
- '==' is the comparison operator → compares two values and returns True or False.

```
a = 10
b = 10
print(a == b) # True
```

⑧ What is a Comment in Python?

Answer: Comments are used to explain the code. Python supports single-line and multi-line comments.

```
# This is a single-line comment
```

```
"""
This is a
multi-line comment
"""
```

⑨ What is the difference between / and // operators?

Answer: / → Division operator (returns float value)
// → Floor division operator (returns integer part)

```
7 / 2 → 3.5
7 // 2 → 3
```

19

⑩ What is a function in Python?

Answer: A function is a block of code that performs a specific task. It helps in code reusability and makes the program modular.

```
def add(a, b):
    return a + b
add(3, 4) # 7
```



PYTHON INTERVIEW NOTES



TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



① What is Python?

Answer: Python is a high-level, interpreted, object-oriented programming language. It is easy to learn, has simple syntax and a vast standard library.

② What are the key features of Python?

Answer:

- Easy to read and write
- Interpreted language
- Object-oriented
- Platform independent
- Open source
- Large standard library
- Supports multiple programming paradigms (OOP, Functional, Procedural)

③ What is the difference between a compiled language and an interpreted language?

Answer:

Compiled Language	Interpreted Language
• Converts entire program into machine code before execution. • Executes faster. • Examples: C, C++	• Executes code line by line. • Easier to debug. • Examples: Python

④ What are variables in Python?

Answer: A variable is a name used to store data in memory. Python automatically determines the data type based on the assigned value.

```
name = "Mounika"
age = 22
cgpa = 8.5
is_student = True
```

⑤ What are the built-in data types in Python?

Answer:

int float str bool list tuple set dict complex

⑥ What is the difference between a List and a Tuple?

Answer:

List

- Mutable (can be changed)
 - Uses square brackets []
- Example: my_list = [1, 2, 3]

Tuple

- Immutable (cannot be changed)
 - Uses parentheses ()
- Example: my_tuple = (1, 2, 3)

⑦ What is the difference between '=' and '=='?

Answer:

- '=' is the assignment operator → assigns a value to a variable.
- '==' is the comparison operator → compares two values and returns True or False.

```
a = 10
b = 10
print(a == b) # True
```

⑧ What is a Comment in Python?

Answer: Comments are used to explain the code. Python supports single-line and multi-line comments.

This is a single-line comment

```
"""
This is a
multi-line comment
"""
```

⑨ What is the difference between / and // operators?

Answer: / → Division operator (returns float value)

// → Floor division operator (returns integer part)

```
7 / 2 → 3.5
7 // 2 → 3
```

⑩ What is a function in Python?

Answer: A function is a block of code that performs a specific task. It helps in code reusability and makes the program modular.

```
def add(a, b):
    return a + b
add(3, 4) # 7
```

Practice Daily ☆ Stay Consistent ☆ Crack Your Dream Job!



PYTHON INTERVIEW NOTES

TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



11 What is the difference between deep copy and shallow copy?

Answer:

- Shallow copy copies the reference. Changes in the original object may reflect in the copied object.
- Deep copy creates a new object and recursively copies all nested objects.

```
import copy
a = [1, 2, [3, 4]]
b = copy.copy(a)    # Shallow copy
c = copy.deepcopy(a) # Deep copy
```

12 What are *args and **kwargs in Python?

Answer:

- *args allows a function to accept any number of positional arguments as a tuple.
- **kwargs allows a function to accept any number of keyword arguments as a dictionary.

```
def func(*args, **kwargs):
    print(args)
    print(kwargs)
func(1, 2, 3, name="Mounika", age=22)
```

13 What is a lambda function in Python?

Answer: A lambda function is a small anonymous function. It can have any number of arguments but only one expression.

```
add = lambda x, y: x + y
print(add(5, 3)) # 8
```

14 What is a docstring in Python?

Answer: Docstring is a string literal that appears right after the definition of a function, method, class or module. It describes what the object does.

```
def square(x):
    """This function returns
    the square of a number."""
    return x * x
help(square)
```

15 What is the difference between remove(), discard() and pop() in a set?

Method	Description	Raises Error if Not Found?	Returns
remove()	Removes the specified element	Yes (KeyError)	None
discard()	Removes the specified element	No	None
pop()	Removes and returns an arbitrary element	Yes (KeyError if set is empty)	The removed element

16 What is the Global Interpreter Lock (GIL) in Python?

Answer: GIL is a mutex that protects access to Python objects, preventing multiple native threads from executing Python bytecodes at once. It affects multithreading performance in Python.



17 What are Python decorators?

Answer: Decorators are functions that modify the behavior of another function or method. They are written using @decorator_name syntax.

```
@my_decorator
def greet():
    print("Hello!")
```

18 What is the difference between == and is?

Answer:

- == compares the values of two objects.
- is checks if two variables point to the same object in memory.

```
a = [1, 2]
b = [1, 2]
print(a == b) # True
print(a is b) # False
```

19 What is a generator in Python?

Answer: A generator is a function that returns an iterator that produces a sequence of values lazily (one at a time) using the yield keyword.

```
def count(n):
    i = 1
    while i <= n:
        yield i
        i += 1
```

20 What is the difference between range() and xrange()?

Answer:

- range() returns a list-like sequence (in Python 3 it returns a range object).
- xrange() was used in Python 2, it returns an iterator and is more memory efficient.

```
range(5)    # range(0, 5)
# xrange() is not used in Python 3
```



PYTHON INTERVIEW NOTES

TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



21) What is the difference between lists and dictionaries?

Answer:

- List is an ordered collection of items.
- Dictionary is an unordered collection of key-value pairs.
- Lists are indexed by position, dictionaries are indexed by keys.

```
my_list = [1, 2, 3]
my_dict = {'name': 'Mounika',
           'age': 22, 'city': 'Tirupati'}
```

22) What is the difference between append() and extend() in a list?

Answer:

- append() adds a single element to the end of the list.
- extend() adds all elements of an iterable to the end of the list.

```
a = [1, 2, 3]
a.append(4)      # [1, 2, 3, 4]
a.extend([5, 6]) # [1, 2, 3, 4, 5, 6]
```

23) What is the difference between break, continue and pass?

Answer:

- **break**: Terminates the loop completely.
- **continue**: Skips the current iteration and continues with next.
- **pass**: Does nothing. It is a placeholder for future code.

```
for i in range(5):
    if i == 2:
        continue
    elif i == 4:
        break
    else:
        pass
```

24) What is the difference between is and == in Python?

Answer:

- == checks value equality.
- is checks identity (same memory location).

```
a = [1, 2]
b = [1, 2]
c = a
print(a == b) # True
print(a is b) # False
print(a is c) # True
```

25) What is a module in Python?

Answer: A module is a file containing Python code (functions, variables, classes etc.) which can be imported in other Python programs to reuse the code.

```
# mymodule.py
def greet():
    return "Hello!"

import mymodule
print(mymodule.greet())
```

26) What is the purpose of the 'self' parameter in Python classes?

Answer: 'self' refers to the instance of the class. It is used to access variables and methods of the class within the class.

```
class Student:
    def __init__(self, name):
        self.name = name
    def show(self):
        print(self.name)
```

27) What is inheritance in Python?

Answer: Inheritance is a way to form new classes using classes that have already been defined. It promotes code reusability.

```
class Parent:
    def show(self):
        print("Parent")
class Child(Parent):
    pass
c = Child()
c.show() # Parent
```

28) What is polymorphism in Python?

Answer: Polymorphism means 'many forms'. It allows objects of different classes to be treated as objects of a common base class.

```
class Bird:
    def sound(self): pass
class Dog(Bird):
    def sound(self):
        print("Bark")
Dog().sound() # Bark
```

29) What is encapsulation in Python?

Answer: Encapsulation is the bundling of data (variables) and methods that work on the data into a single unit (class).

```
class Account:
    def __init__(self, bal):
        self.__balance = bal # private
    def get_balance(self):
        return self.__balance
```

30) What is a constructor in Python?

Answer: A constructor is a special method "__init__" which is automatically called when an object of a class is created.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
p = Person("Mounika", 22)
```




PYTHON INTERVIEW NOTES



TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



31) What is the difference between a shallow copy and a deep copy?

Answer:

- Shallow copy copies the reference of object. Changes in the original object may reflect in the copy.
- Deep copy copies the actual object and all nested objects. Changes in the original object do not affect the copy.

```
import copy
a = [1, [2, 3]]
b = copy.copy(a)      # shallow copy
c = copy.deepcopy(a)   # deep copy
```

32) What are iterators in Python?

Answer: An iterator is an object that contains a countable number of values. It can be iterated upon using the iter() and next() functions.

```
my_list = [10, 20, 30]
it = iter(my_list)
print(next(it))  # 10
print(next(it))  # 20
print(next(it))  # 30
```

33) What is the difference between list comprehension and normal for loop?

Answer:

- List comprehension is more concise and readable.
- For loop is more flexible and easy to understand for beginners.

```
# List comprehension
squares = [x*x for x in range(5)]

# For loop
squares = []
for x in range(5):
    squares.append(x*x)
```

34) What is a Python package?

Answer: A package is a collection of Python modules and sub-packages organized in a directory hierarchy. It helps in modularizing the code.

```
mypackage/
__init__.py
module1.py
module2.py
subpackage/
__init__.py
module3.py
```

35) What is the purpose of the '__init__.py' file?

Answer: It is used to mark a directory as a Python package. It can also contain initialization code for the package.

```
# __init__.py can be empty
# or can include initialization code
```

36) What are generators in Python?

Answer: Generators are a simple way of creating iterators using a function with the 'yield' keyword. They are memory efficient.

```
def count(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for num in count(5):
    print(num)
```

37) What is the difference between map(), filter() and reduce()?

Answer:

- map(): Applies a function to all items in an iterable.
- filter(): Filters items that return True for a function.
- reduce(): Applies a function cumulatively to the items.

```
from functools import reduce
nums = [1, 2, 3, 4]
list(map(lambda x: x*2, nums))  # [2, 4, 6, 8]
list(filter(lambda x: x%2==0, nums))  # [2, 4]
reduce(lambda x, y: x+y, nums)  # 10
```

38) What is the difference between a function and a method?

Answer:

- Function: A block of code that performs a task.
- Method: A function that is associated with an object. It is called on an object.

```
# Function
def add(a, b):
    return a + b

# Method
"""upper() is a method of string class"""
text = "mounika"
print(text.upper())  # MOUNIKA
```

39) What is the difference between mutable and immutable objects?

Answer:

- Mutable: Objects whose state or value can be changed.
- Immutable: Objects whose state or value cannot be changed.

```
# Mutable
my_list = [1, 2, 3]
my_list[0] = 10  # Can be changed

# Immutable
my_tuple = (1, 2, 3)
# my_tuple[0] = 10  # Error
```

40) What is a docstring in Python?

Answer: Docstring is a string literal that appears right after the definition of a module, function, class or method. It describes what the object does.

```
def greet(name):
    """This function greets the person
    whose name is passed as argument."""
    return f"Hallo, {name}!"

print(greet.__doc__)  # Prints docstring
```



PYTHON INTERVIEW NOTES



TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS

41) What is the purpose of the 'super()' function?

Answer:

- super() is used to call methods from the parent class.
- It is commonly used in inheritance.

```
class Parent:
    def show(self):
        print("Parent method")

class Child(Parent):
    def show(self):
        super().show() # Call parent method
        print("Child method")

c = Child()
c.show()
```

42) What is isinstance() in Python?

Answer: isinstance() checks whether an object is an instance of a class or a tuple of classes.

```
x = 10
print(isinstance(x, int)) # True
print(isinstance(x, str)) # False
print(isinstance(10, (int, float))) # True
```

43) What is the difference between "+" and "*" operators in strings and lists?

Answer:

- + is used for concatenation.
- * is used for repetition.

```
# Strings
s = "Hello" + " World" # Concatenation
s2 = "Hi" * 3           # Repetition

# Lists
lst = [1, 2] + [3, 4]  # Concatenation
lst2 = [1] * 3          # Repetition
```

44) What is a lambda expression in Python?

Answer: A lambda expression is a small anonymous function without a name. It is created using the 'lambda' keyword.

```
add = lambda a, b: a + b
print(add(5, 3)) # 8
```

45) What is the purpose of the 'enumerate()' function?

Answer: enumerate() adds a counter to an iterable and returns it as an enumerate object (index, value).

```
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(index, fruit)

# Output: 0 apple
#         1 banana
#         2 cherry
```

46) What is the purpose of the 'zip()' function?

Answer: zip() combines multiple iterables element-wise and returns an iterator of tuples.

```
names = ['Mounika', 'Ravi', 'Neha']
ages = [22, 24, 21]
z = zip(names, ages)
print(list(z))
# [('Mounika', 22), ('Ravi', 24), ('Neha', 21)]
```

47) What are *args and **kwargs?

Answer:

- *args allows a function to accept any number of positional arguments.
- **kwargs allows a function to accept any number of keyword arguments.

```
def func(*args, **kwargs):
    print("args:", args)
    print("kwargs:", kwargs)

func(1, 2, 3, name="Mounika", city="Tirupati")
# args: (1, 2, 3)
# kwargs: {'name': 'Mounika', 'city': 'Tirupati'}
```

48) What is the difference between 'is' and '=='?

- == compares the values of two objects.
- is checks if both variables point to the same object in memory.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b) # True
print(a is b) # False
print(a is c) # True
```

49) What is a set in Python?

Answer: A set is an unordered collection of unique elements. It does not allow duplicate values.

```
s = {1, 2, 3, 3, 2}
print(s) # {1, 2, 3}
```

50) What is the difference between list and set?

Answer:

Feature	List	Set
Ordered	Yes	No
Duplicate Elements	Allowed	Not Allowed
Syntax	[]	{}
Indexing	Supported	Not Supported
Use Case	When order matters	When uniqueness is required

```
lst = [1, 2, 2, 3]
st = {1, 2, 2, 3}
print(lst) # [1, 2, 2, 3]
print(st) # {1, 2, 3}
```




PYTHON INTERVIEW NOTES

TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



51) What is the difference between tuple and set?

Answer:

Feature	Tuple	Set
Ordered	Yes	No
Mutable	No (Immutable)	Yes (Mutable)
Duplicate Elements	Allowed	Not Allowed
Syntax	()	{}
Indexing	Supported	Not Supported
Use Case	Heterogeneous data, fixed collection	Unique items, mathematical operations

```
t = (1, 2, 2, 3)
s = {1, 2, 2, 3}
print(t)    # (1, 2, 2, 3)
print(s)    # {1, 2, 3}
```

52) What are the different types of loops in Python?

Answer:

- for loop - used to iterate over a sequence (list, tuple, string, etc.)
- while loop - used to repeat a block of code while a condition is True.

```
# for loop
for i in range(3):
    print(i)    # 0 1 2

# while loop
i = 0
while i < 3:
    print(i)
    i += 1    # 0 1 2
```

53) What is the difference between range() and xrange()?

Answer: • range() returns a list in Python 2 (and a range object in Python 3).

- xrange() returns an iterator in Python 2 (memory efficient).

Note: xrange() is not available in Python 3.

```
# Python 2
range(5)    # [0, 1, 2, 3, 4]
xrange(5)    # xrange(5)

# Python 3
range(5)    # range(0, 5)
```

54) What is the purpose of the 'pass' statement?

Answer: The pass statement is a null operation. It is used as a placeholder for future code.

```
for i in range(5):
    pass    # placeholder

def func():
    pass    # placeholder
```

55) What is the difference between append() and insert() methods in lists?

Answer: • append() adds an element at the end of the list.

- insert() adds an element at the specified index.

```
lst = [1, 2, 3]
lst.append(4)    # [1, 2, 3, 4]
lst.insert(1, 10)    # [1, 10, 2, 3, 4]
```

56) What is the purpose of the 'join()' method in strings?

Answer: join() is used to join elements of an iterable (like list) into a single string using a separator.

```
words = ["Python", "is", "awesome"]
result = " ".join(words)
print(result)    # Python is awesome
```

57) What is the difference between remove(), pop() and del?

Answer:

- remove() removes the first occurrence of the specified element.
- pop() removes and returns the element at the specified index (default is last element).
- del deletes the element at the specified index or the entire list.

```
lst = [10, 20, 30, 40]
lst.remove(20)    # [10, 30, 40]
lst.pop()    # 40, list -> [10, 30]
lst.pop(0)    # 10, list -> [30]
del lst[0]    # []
```

58) What is the difference between break, continue and pass?

Answer:

- break: Terminates the loop completely.
- continue: Skips the current iteration and continues with the next.
- pass: Does nothing. It is a placeholder.

```
for i in range(5):
    if i == 2:
        continue    # skip 2
    if i == 4:
        break    # stop at 4
    print(i)    # 0 1 3
```

59) What is a dictionary in Python?

Answer: A dictionary is an unordered collection of key-value pairs. Keys must be unique and immutable.

```
d = {'name': 'Mounika', 'age': 22, 'city': 'Tirupati'}
print(d['name'])    # Mounika
print(d.keys())    # dict_keys(['name', 'age', 'city'])
print(d.values())    # dict_values(['Mounika', 22, 'Tirupati'])
```

60) What is the difference between keys(), values() and items() in dictionaries?

Answer:

- keys() returns all keys.
- values() returns all values.
- items() returns all key-value pairs as tuples.

```
d = {'a': 1, 'b': 2}
print(d.keys())    # dict_keys(['a', 'b'])
print(d.values())    # dict_values([1, 2])
print(d.items())    # dict_items([('a', 1), ('b', 2)])
```



PYTHON INTERVIEW NOTES

TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



61) What is the difference between a for loop and a while loop?

Answer:

- for loop is used when the number of iterations is known. It iterates over a sequence (list, tuple, string, etc.).
- while loop is used when the number of iterations is not known in advance and depends on a condition.

```
# for loop
for i in range(3):
    print(i)    # 0 1 2

# while loop
i = 0
while i < 3:
    print(i)
    i += 1     # 0 1 2
```

62) What is the purpose of the 'in' operator?

Answer: The 'in' operator checks whether a value exists in a sequence (list, tuple, string, set, dictionary, etc.).

```
lst = [10, 20, 30]
print(20 in lst)    # True
print(40 in lst)    # False

s = "Python"
print('P' in s)     # True
```

63) What is the difference between list(), tuple() and set() type casting?

Answer:

- list() converts an iterable into a list.
- tuple() converts an iterable into a tuple.
- set() converts an iterable into a set (unique elements only).

```
l = list([1, 2, 3]) # [1, 2, 3]
t = tuple([1, 2, 3]) # (1, 2, 3)
s = set([1, 2, 2, 3]) # {1, 2, 3}
```

64) What are Python's built-in functions?

Answer: Some commonly used built-in functions are: print(), len(), type(), input(), int(), float(), str(), range(), max(), min(), sum(), sorted() etc.

```
print(len([1, 2, 3]))    # 3
print(max([4, 7, 2]))    # 7
print(sorted([3, 1, 2])) # [1, 2, 3]
```

65) What is the purpose of the 'enumerate()' function?

Answer: enumerate() adds a counter to an iterable and returns it as an enumerate object.

```
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(index, fruit)    # 0 apple
                        # 1 banana
                        # 2 cherry
```

66) What is the purpose of the 'any()' and 'all()' functions?

Answer:

- any() returns True if at least one element in the iterable is True.
- all() returns True only if all elements in the iterable are True.

```
a = [True, False, False]
b = [True, True, True]
print(any(a))    # True
print(all(a))    # False
print(all(b))    # True
```

67) What is the difference between copy() and deepcopy()?

Answer:

- copy() creates a shallow copy of the object. Nested objects are not copied.
- deepcopy() creates a deep copy of the object. Nested objects are also copied.

```
import copy
x = [[1, 2], [3, 4]]
y = copy.copy(x)
z = copy.deepcopy(x)
x[0][0] = 99
print(y)    # [[99, 2], [3, 4]] Shallow copy changed
print(z)    # [[1, 2], [3, 4]] Deep copy unchanged
```

68) What is a lambda function?

Answer: A lambda function is a small anonymous function defined using the 'lambda' keyword. It can have any number of arguments but only one expression.

```
add = lambda a, b: a + b
print(add(5, 3))    # 8

square = lambda x: x * x
print(square(4))    # 16
```

69) What is the purpose of the 'filter()' function?

Answer: filter() filters elements from an iterable for which a function returns True.

```
nums = [1, 2, 3, 4, 5, 6]
even = list(filter(lambda x: x % 2 == 0, nums))
print(even)    # [2, 4, 6]
```

70) What is the purpose of the 'map()' function?

Answer: map() applies a given function to all items in an iterable (list, tuple, etc.) and returns a map object.

```
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x*x, nums))
print(squares)    # [1, 4, 9, 16]
```



Keep Practicing, Keep Growing!





PYTHON INTERVIEW NOTES

TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



71 What is the purpose of the 'reduce()' function?

Answer: reduce() applies a rolling computation to sequential pairs of values in an iterable and returns a single result.

```
from functools import reduce
nums = [1, 2, 3, 4]
product = reduce(lambda x, y: x * y, nums)
print(product)    # 24
```

72 What are list comprehensions in Python?

Answer: List comprehensions provide a concise way to create lists using an expression inside square brackets.

```
squares = [x**2 for x in range(5)]
print(squares)    # [0, 1, 4, 9, 16]
```

73 What is a Python decorator?

Answer: A decorator is a function that takes another function and extends the behavior of the latter function without permanently modifying it.

```
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")

say_hello()
```

74 What is the Global Interpreter Lock (GIL) in Python?

Answer: GIL is a mutex that protects access to Python objects, preventing multiple native threads from executing Python bytecodes at once in CPython.

```
# Due to GIL, Python threads are best
# for I/O-bound tasks, not CPU-bound tasks.
```

75 What is the difference between a function and a lambda function?

Answer:

- Function is defined using 'def' keyword and can have multiple statements.
- Lambda function is an anonymous function with only one expression.

```
# Function
def add(a, b):
    return a + b

# Lambda function
add = lambda a, b: a + b
print(add(5, 3))    # 8
```

76 What is the purpose of the 'property' decorator?

Answer: The property decorator allows the definition of methods that can be accessed like attributes.

```
class Person:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name

p = Person("Alice")
print(p.name)    # Alice
```

77 What is the difference between mutable and immutable types with examples?

Answer:

- Mutable types can be changed after creation.
Examples: list, dict, set
- Immutable types cannot be changed after creation.
Examples: int, float, str, tuple

```
# Mutable
lst = [1, 2, 3]
lst.append(4)    # [1, 2, 3, 4]

# Immutable
s = "hello"
# s[0] = 'H'    # Error: strings are immutable
```

78 What is the purpose of the 'with' statement?

Answer: The 'with' statement is used to wrap the execution of a block of code within methods defined by a context manager. It ensures proper acquisition and release of resources.

```
with open('file.txt', 'r') as f:
    content = f.read()    # file is automatically
    print(content)        # closed after block
```

79 What is the difference between a shallow copy and a deep copy?

Answer:

- Shallow copy creates a new object but inserts references into it to the objects found in the original.
- Deep copy creates a new object and recursively adds copies of nested objects found in the original.

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)    # shallow copy
c = copy.deepcopy(a)    # deep copy

a[0][0] = 99
print(b)    # [[99, 2], [3, 4]]
print(c)    # [[1, 2], [3, 4]]
```

80 What is the purpose of the '__str__()' and '__repr__()' methods?

Answer:

- __str__() is used for user-friendly (readable) output.
- __repr__() is used for developer-friendly (unambiguous) representation.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self):
        return f"Point({self.x}, {self.y})"
    def __repr__(self):
        return f"Point(x={self.x}, y={self.y})"
```



PYTHON INTERVIEW NOTES

TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



81) What is the purpose of the '@staticmethod' decorator?

Answer: The @staticmethod decorator defines a method

- that belongs to the class but does not modify the class or instance.
- It does not take 'self' or 'cls' as the first parameter.

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b

print(Math.add(3, 5)) # 8
```

82) What is a generator in Python?

Answer: A generator is a function that returns an iterator which generates items one at a time using 'yield'.

```
def count(n):
    i = 1
    while i <= n:
        yield i
        i += 1

for num in count(3):
    print(num) # 1 2 3
```

83) What is the purpose of the 'super()' function?

Answer: super() is used in a child class to call methods from its parent class.

```
class A:
    def show(self):
        print("Parent class")

class B(A):
    def show(self):
        super().show()
        print("Child class")

b = B() # Parent class
b.show() # Child class
```

84) What is the purpose of the '__del__()' method in Python?

Answer: The __del__() method is called when an object is about to be destroyed. It is used for cleanup.

```
class Demo:
    def __del__(self):
        print("Object is destroyed")

obj = Demo()
del obj # Object is destroyed
```

85) What is the difference between 'is' and '=='?

Answer:

- 'is' checks if two variables point to the same object.
- '==' checks if the values of two objects are equal.

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b) # True
print(a is b) # False
print(a is c) # True
```

86) What is the purpose of the 'zip_longest()' function?

Answer: zip_longest() returns an iterator of tuples, similar to zip(), but fills missing values with a specified value.

```
from itertools import zip_longest
a = [1, 2, 3]
b = ['a', 'b']
for x, y in zip_longest(a, b, fillvalue='-'):
    print(x, y) # 1 a / 2 b / 3 -
```

87) What is the purpose of the 'sorted()' function?

Answer: sorted() returns a new sorted list from the elements of any iterable without modifying the original.

```
nums = [3, 1, 4, 2]
print(sorted(nums)) # [1, 2, 3, 4]
print(sorted(nums, reverse=True)) # [4, 3, 2, 1]
```

88) What is the purpose of the 'any()' and 'all()' functions with examples?

Answer:

- any() returns True if at least one element is True.
- all() returns True if all elements are True.

```
print(any([False, False, True])) # True
print(any([False, False])) # False
print(all([True, True, True])) # True
print(all([True, False, True])) # False
```

89) What is a context manager in Python?

Answer: A context manager is an object that defines the runtime context to be established when executing a block of code using 'with' statement.

```
with open('file.txt', 'r') as f:
    data = f.read() # File is automatically
                    # closed after block
```

90) What is the purpose of the 'breakpoint()' function?

Answer: breakpoint() is used for debugging. It drops the program into the debugger at that point.

```
def add(a, b):
    result = a + b
    breakpoint() # Execution will pause here
    return result
```




PYTHON INTERVIEW NOTES

TOP 100 FREQUENTLY ASKED PYTHON INTERVIEW QUESTIONS



91 What is the purpose of the 'defaultdict' in Python?

Answer: defaultdict is a dictionary subclass that provides a default value for a missing key.

```
from collections import defaultdict
d = defaultdict(int)
d['a'] += 1
print(d['a']) # 1 (key 'a' did not exist before)
```

92 What is the purpose of the 'isinstance()' function?

Answer: isinstance() checks whether an object is an instance of a specified class or a subclass.

```
x = 10
print(isinstance(x, int)) # True
print(isinstance(x, (int, float))) # True
```

93 What is the difference between 'pop()' and 'popitem()' in dictionary?

Answer:

- pop(key) removes and returns the value of the specified key.
- popitem() removes and returns the last inserted key-value pair.

```
d = {'a': 1, 'b': 2, 'c': 3}
print(d.pop('b')) # 2
print(d) # {'a': 1, 'c': 3}
print(d.popitem()) # ('c', 3)
print(d) # {'a': 1}
```

94 What is the purpose of the 'enumerate()' function with start parameter?

Answer: enumerate(iterable, start) returns index-value pairs starting from the specified start value.

```
fruits = ["apple", "banana", "cherry"]
for idx, fruit in enumerate(fruits, start=1):
    print(idx, fruit)

# Output:
# 1 apple
# 2 banana
# 3 cherry
```

95 What is the purpose of the 'bin()', 'oct()' and 'hex()' functions?

Answer: These functions convert an integer into its binary, octal and hexadecimal string representation.

```
n = 25
print(bin(n)) # 0b11001
print(oct(n)) # 0o31
print(hex(n)) # 0x19
```

96 What is the difference between '__getattr__()' and '__getattribute__()'?

Answer:

- __getattribute__() is called for every attribute access.
- __getattr__() is called only if the attribute is not found.

```
class Demo:
    def __getattr__(self, name):
        return f"{name} not found"

d = Demo()
print(d.unknown) # unknown not found
```

97 What is the purpose of the 'itertools' module?

Answer: itertools provides fast, memory-efficient tools for creating iterators for loops.

```
from itertools import combinations
data = [1, 2, 3]
print(list(combinations(data, 2)))
# [(1, 2), (1, 3), (2, 3)]
```

98 What is the purpose of the 'secrets' module?

Answer: secrets is used to generate cryptographically strong random numbers.

```
import secrets
token = secrets.token_hex(16)
print(token) # secure random token
```

99 What is the difference between '==' and 'is'?

Answer:

- '==' checks value equality.
- 'is' checks object identity (same memory location).

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b) # True
print(a is b) # False
print(a is c) # True
```

100 What is the purpose of the 'help()' function?

Answer: help() is used to get the documentation for modules, classes, functions or methods.

```
help(len) # Opens help documentation for len()
```